

Лекция 8. Централизованный сбор логов: Syslog и rsyslog

Цель лекции

Понять, зачем предприятию нужен централизованный сбор системных журналов

Объяснять Syslog через facility, severity, формат сообщения и сетевой транспорт

Различать роли log-клиента, relay-узла и сервера-коллектора

Настраивать базовый прием и пересылку логов через rsyslog

Проверять доставку сообщений, ротацию журналов и типовые ошибки настройки

Учебные вопросы: 1-4

1. Проблема анализа и безопасности локальных журналов на предприятии
2. Архитектура и стандарт Syslog RFC 5424
3. Участники процесса: клиенты, релей и сервер-коллектор
4. Архитектура и конфигурация rsyslog в Linux

Учебные вопросы: 5-8

- 5. Практика: настройка rsyslog-сервера-коллектора
- 6. Практика: настройка rsyslog-клиента-отправителя
- 7. logger и ротация логов logrotate
- 8. Диагностика, тестирование и типичные ошибки

Учебный сценарий лекции

Внутренняя учебная зона: company.test

Сервер log01.company.test с адресом 192.168.10.100
принимает журналы от Linux-серверов

Клиент web01.company.test отправляет свои
системные и служебные события на log01

Журналы удаленных узлов раскладываются в
/var/log/remote/%HOSTNAME%/PROGRAMNAME%.log

Проблема анализа и безопасности локальных журналов на предприятии

Log, журнал событий, фиксирует действия системы, служб, пользователей и ошибок

На одном сервере администратор может посмотреть /var/log вручную

Когда серверов становится 10-20 и больше, ручной просмотр журналов превращается в медленную и неполную проверку

Централизованный сбор логов нужен, чтобы видеть события инфраструктуры в одном месте

Для чего нужны системные журналы

В аудите безопасности журналы помогают понять, кто входил в систему и какие действия выполнял

При сбоях журналы показывают причину остановки службы, ошибку конфигурации или нехватку ресурса

В мониторинге журналы дополняют метрики и показывают текстовые признаки проблемы

Для расследования инцидента важна не одна строка, а последовательность событий во времени

Почему локальных журналов недостаточно

Локальный журнал хранится на том же сервере, который может выйти из строя

Если диск сервера поврежден или переполнен, важные записи могут быть потеряны

Администратор тратит время на подключение к каждому узлу отдельно

Сравнивать события с разных серверов трудно, если время и формат записей не согласованы

Локальные логи и риск после взлома

После компрометации сервера злоумышленник часто пытается удалить следы из `/var/log`

Если журналы уже отправлены на отдельный сервер, удалить следы становится сложнее

Сервер-коллектор должен быть защищен строже, чем обычные прикладные серверы

Централизация не отменяет защиту локального сервера, но повышает шанс сохранить доказательства

Локальные журналы и централизованный сбор логов

16:9

❌ Проблема: локальные логи

- Журналы распределены по серверам
- Администратор подключается к каждому
- При взломе логи могут быть удалены
- Сложно сопоставлять события по времени
- Переполнение диска → потеря данных

web01

HTTP сервер
rsyslog клиент

db01

СУБД сервер
rsyslog клиент

app01

Приложение
rsyslog клиент

Сеть

UDP/TCP 514

Syslog

RFC 5424

log01

rsyslog
collector

UDP 514
TCP 514

/var/log
/remote

✅ Решение: централизованный сбор

- Все журналы в одном месте
- Единая структура хранения
- Защита от удаления следов
- Легкий поиск и корреляция
- Ротация и архивация логов

📁 Структура хранения на коллекторе log01

`/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log`

Динамический шаблон RemoteLogs

Пример:

`/var/log/remote/web01/nginx.log`

📄 Компоненты Syslog-сообщения

Facility

auth, cron, daemon
kern, mail, syslog
local0-local7

Severity

0 Emergency → 7 Debug
Error, Warning, Info
Критичность события

Транспорт

UDP 514 — быстрый
TCP 514 — надёжный
TLS — защищённый

Message

Текст события
Timestamp, Hostname
Понятный администратору

ЛОКАЛЬНЫЕ ЖУРНАЛЫ И ЦЕНТРАЛИЗОВАННЫЙ СБОР

Зачем предприятию нужен централизованный сбор системных журналов

ЛОКАЛЬНЫЕ ЖУРНАЛЫ: ПРОБЛЕМЫ



1. Риск потери данных

Если сервер вышел из строя, журналы теряются вместе с ним



2. Переполнение и повреждение диска

При переполнении или сбое диска важные записи могут быть утрачены



3. Трудоемкий ручной просмотр

Администратор подключается к каждому узлу отдельно и тратит много времени



4. Сложно сравнивать события

Разные серверы, разное время и формат — трудно видеть общую картину



5. Риск после взлома

Злоумышленник может удалить следы из /var/log на скомпрометированном узле



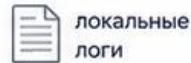
Локальные журналы дают неполную и ненадёжную картину.
Нужен централизованный сбор!

ЦЕНТРАЛИЗОВАННЫЙ СБОР: КАК ЭТО РАБОТАЕТ

ИСТОЧНИКИ СОБЫТИЙ (КЛИЕНТЫ)



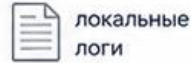
web01.company.test
192.168.10.11



локальные логи



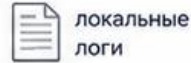
app01.company.test
192.168.10.12



локальные логи



db01.company.test
192.168.10.13



локальные логи

Syslog
(UDP/TCP 514)

СЕРВЕР-КОЛЛЕКТОР

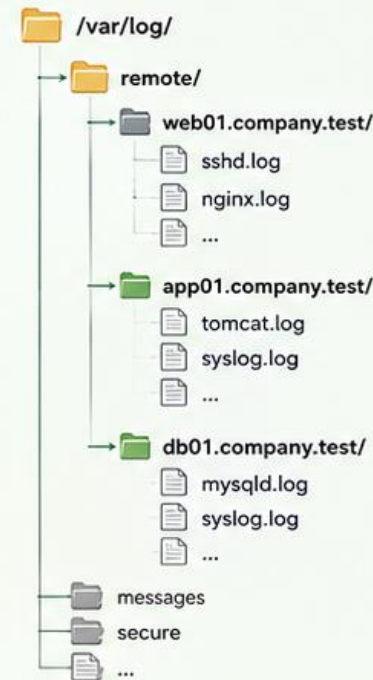
log01.company.test

192.168.10.100



ХРАНИЕНИЕ ЖУРНАЛОВ

Структура на сервере-коллекторе



ЧТО МЫ ПОЛУЧАЕМ



Сохранность журналов даже при сбое узла



Быстрый поиск событий в одном месте



Корреляция событий с разных серверов



Удобство анализа и расследования



Контроль доступа и защита данных



Сценарий из лекции:

Сервер log01.company.test (192.168.10.100) принимает журналы от Linux-серверов. Клиент web01.company.test отправляет свои события на log01. Журналы удалённых узлов сохраняются в /var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log

ЧТО ДОЛЖЕН ОБЕСПЕЧИТЬ LOG-СЕРВЕР



Приём по сети

Принимать события от разных узлов по сети (Syslog)



Понятная структура

Сохранять журналы в удобной структуре каталогов и файлов



Отделение логов

Отделять удалённые логи от собственных локальных журналов сервера



Ресурсы и ротация

Иметь достаточно места на диске и настроенную ротацию (logrotate)



Безопасность

Ограничивать доступ к журналам — это может быть чувствительная информация

Что должен обеспечить log-сервер

Принимать события от разных узлов по сети

Сохранять журналы в понятной структуре каталогов и файлов

Отделять удаленные логи от собственных локальных журналов

Иметь достаточно дискового пространства и настроенную ротацию

Ограничивать доступ к журналам, потому что они могут содержать чувствительные данные

2. Архитектура и стандарт Syslog RFC 5424

Syslog — стандартный подход к передаче и описанию системных сообщений журналирования

Syslog можно понимать как формат сообщения, протокол передачи и сетевой сервис

RFC (Request for Comments) 5424 описывает современный формат Syslog-сообщений

Syslog помогает разным устройствам и службам передавать события в едином виде

Из чего состоит Syslog-сообщение

Facility показывает источник или категорию события

Severity показывает важность или критичность события

Timestamp фиксирует время создания сообщения

Hostname показывает узел, который отправил сообщение

Message содержит текст события, понятный администратору или системе анализа

Facility: источник сообщения

Facility, источник сообщения, помогает отделять события безопасности, ядра, почты и приложений
auth и authpriv применяются для событий авторизации и безопасности

cron используется для событий планировщика заданий

daemon применяется для системных служб и демонов
local0-local7 оставлены для пользовательских приложений и учебных сценариев

Severity: уровень важности

Severity, уровень важности, определяет срочность реакции администратора

Emergency, Alert и Critical означают аварийные или близкие к аварийным состояния

Error и Warning показывают ошибки и предупреждения, требующие анализа

Notice и Informational описывают нормальные, но полезные события

Debug содержит отладочную информацию и может быстро создавать большой объем логов

Шкала severity 0-7

- 0 Emergency: система неработоспособна
- 1 Alert: требуется немедленное вмешательство
- 2 Critical и 3 Error: критическое состояние или ошибка службы
- 4 Warning и 5 Notice: предупреждение или значимое нормальное событие
- 6 Informational и 7 Debug: информационные и отладочные сообщения

Facility, severity и тело Syslog-сообщения

16:9

Структура Syslog-сообщения (RFC 5424)

<13> 2024-01-15T10:30:45 web01 sshd[1234] Failed password for root from 192.168.1.100

PRI Timestamp Hostname Tag (Program + PID) Message Body

Facility (источник)

Код	Facility	Код	Facility
0	kern	7	news
1	user	8	uucp
2	mail	9	cron
3	daemon	10	authpriv
4	auth	11	ftp
5	syslog	16-23	local0-7
6	lpr	★	Пользовательские

Severity (важность)

0	1	2	3	4	5	6	7
Emergency	Alert	Critical	Error	Warning	Notice	Info	Debug
0-1: Система неработоспособна / Немедленное вмешательство		2-4: Критическое состояние / Ошибка / Предупреждение		5-7: Заметное событие / Информация / Отладка			

★ Чем выше значение — тем ниже важность

$$PRI = (Facility \times 8) + Severity$$

Пример: Facility=1 (user), Severity=5 (notice) → PRI=13 → <13>

Пример разбора

Исходное сообщение

<13>2024-01-15T10:30:45 web01 sshd[1234]:

<13> → Facility=1 (user), Severity=5 (notice)

2024-01-15T10:30:45 → Timestamp (ISO 8601)

web01 → Hostname (отправитель)

sshd[1234] → Program + PID

Failed password for root... → Message Body

Ключевые моменты

1

Facility определяет **источник** события (kern, mail, cron, local0 и др.)
Severity определяет **важность** события (0 — Emergency, 7 — Debug)

2

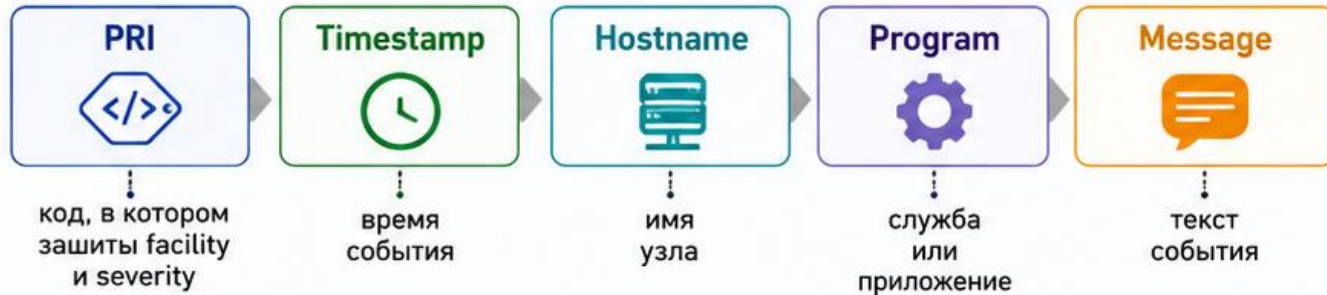
PRI вычисляется по формуле: **(Facility × 8) + Severity**
PRI помещается в угловые скобки: <PRI> в начале сообщения

★ Практическое применение: фильтрация по facility (auth — безопасность) и severity (Error — ошибки)
Правильная классификация помогает быстро находить критичные события в потоке логов

FACILITY, SEVERITY И ТЕЛО SYSLOG-СООБЩЕНИЯ

Из чего состоит Syslog-сообщение и как понять его важность

1 ЧТО ВХОДИТ В SYSLOG-СООБЩЕНИЕ



>_ <34> Oct 11 10:15:22 web01 sshd: Failed password for admin from 192.168.10.15

2 FACILITY — ИСТОЧНИК СООБЩЕНИЯ

Facility показывает, откуда пришло событие



auth/authpriv
аутентификация
и безопасность



cron
планирующих
заданий



daemon
системные
службы



kern
ядро
Linux



mail
почтовые
службы



local0–local7
пользовательские
приложения

3 SEVERITY — УРОВЕНЬ ВАЖНОСТИ

Severity показывает срочность реакции

КРИТИЧНО ↑ ↓ ОБЫЧНО	0	Emergency	система неработоспособна
	1	Alert	нужно немедленное действие
	2	Critical	критическая проблема
	3	Error	ошибка
	4	Warning	предупреждение
	5	Notice	значимое нормальное событие
	6	Informational	информационное сообщение
	7	Debug	отладка

4 КАК ЧИТАЕТСЯ PRI

$$\text{PRI} = \text{Facility} \times 8 + \text{Severity}$$

Facility (источник) 4 × 8 + Severity (важность) 2 = 34 PRI



Если facility = 4 (auth) и severity = 2 (critical), то PRI = 4×8+2 = 34

5 КАК АДМИНИСТРАТОР ИСПОЛЬЗУЕТ ЭТУ ИНФОРМАЦИЮ



Фильтрует логи по источнику



Понимает важность события



Быстрее ищет инциденты



Настраивает пересылку и правила в rsyslog



Facility отвечает на вопрос:
ОТКУДА?



Severity отвечает на вопрос:
НАСКОЛЬКО ВАЖНО?



Message отвечает на вопрос:
ЧТО СЛУЧИЛОСЬ?

UDP 514 и TCP 514

UDP (User Datagram Protocol) 514 — быстрый транспорт Syslog без подтверждения доставки
TCP (Transmission Control Protocol) 514 использует соединение и надежнее при передаче важных событий

UDP подходит для простых сценариев и большого потока некритичных сообщений

TCP предпочтительнее, когда потеря журналов недопустима или требуется буферизация на клиенте
Firewall на log-сервере должен разрешать выбранный протокол и порт

3. Участники процесса: клиенты, релей и сервер-коллектор

Log Generator, клиент или генератор логов, создает события и отправляет их наружу

Log Relay, релей, принимает события из сегмента сети и пересылает их дальше

Log Collector, сервер-коллектор, принимает и хранит журналы от многих узлов

Разделение ролей помогает строить логирование для филиалов, DMZ и разных подсетей

Log Generator: источник событий

Клиентом может быть Linux-сервер, маршрутизатор, firewall, веб-сервер или приложение

На Linux роль отправителя часто выполняет rsyslog

Клиент должен знать адрес log-сервера, транспорт и правило пересылки

На web01.company.test клиент будет отправлять все события на 192.168.10.100

Log Relay: промежуточный узел

Релей полезен, когда филиал имеет много устройств и узкий канал до центрального офиса

Он может агрегировать события локального сегмента и пересылать их на главный коллектор

Релей снижает число прямых соединений между филиалом и центральным log-сервером

Если релей настроен ошибочно, можно получить задержки или потерю части событий

Log Collector: сервер хранения журналов

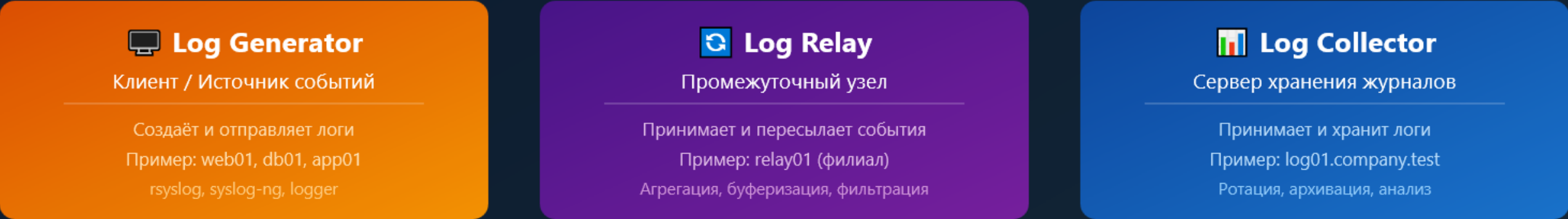
Коллектор принимает сообщения, раскладывает их по файлам и хранит историю событий

Ему нужны диск, ротация, резервное копирование и ограничение доступа

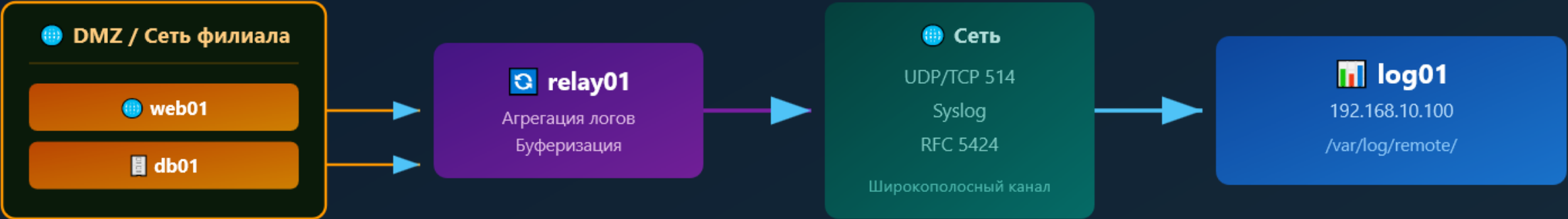
На log01 удаленные события будут храниться отдельно от локальных журналов самого сервера

Сервер-коллектор не должен бесконтрольно пересылать принятые события обратно клиентам

Клиент, релей и сервер-коллектор в Syslog-инфраструктуре



Топология с релеем

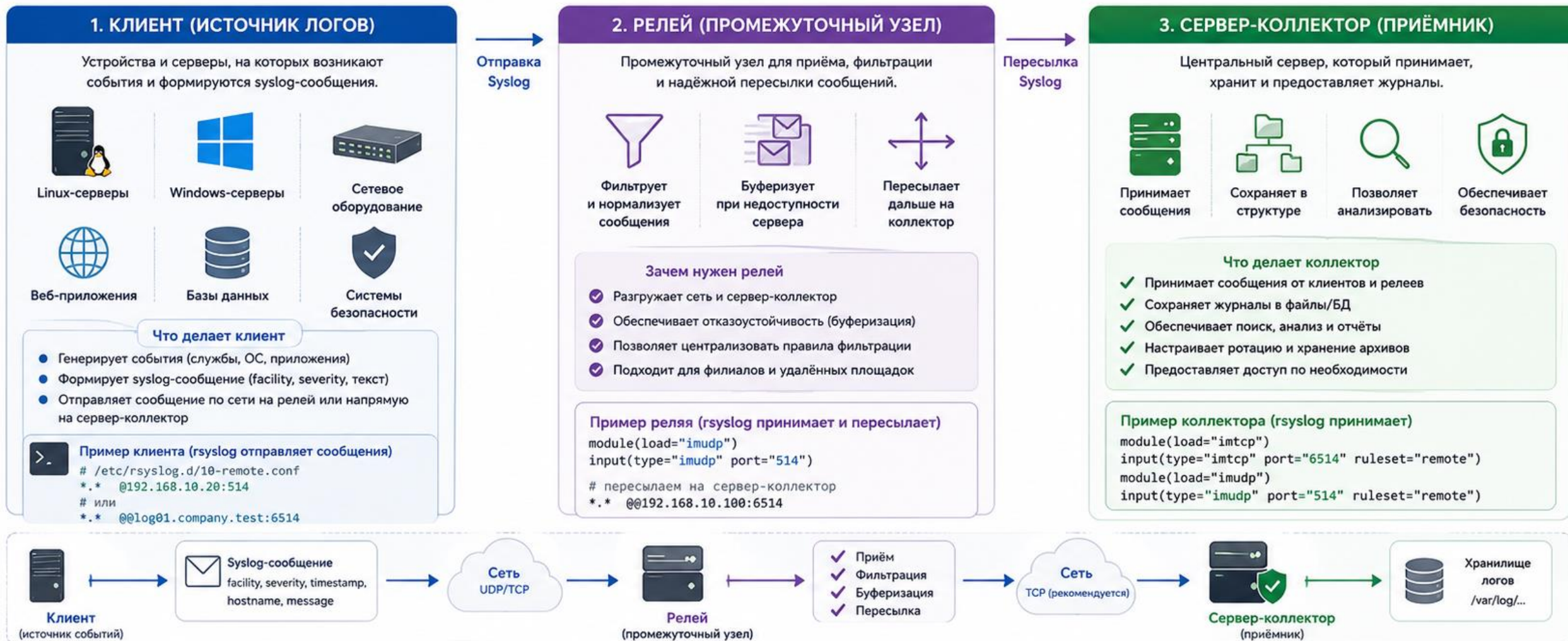


Сравнение ролей

Роль	Задача	Пример	Особенности	Транспорт
 Клиент (Generator)	Создаёт и отправляет системные события	web01, db01 app01, firewall	Может иметь очередь на случай недоступности	UDP/TCP 514 TLS (опционально)
 Релей (Relay)	Принимает, агрегирует и пересылает события	relay01 (филиал/сегмент)	Снижает число соединений Буферизация на relay	TCP (надёжный) TLS (защищённый)

КЛИЕНТ, РЕЛЕЙ И СЕРВЕР-КОЛЛЕКТОР В SYSLOG

Как путешествуют системные журналы от источника до центра хранения



КАКИЕ ПРОТОКОЛЫ ИСПОЛЬЗУЕМ

- UDP/514** → Просто и быстро, но без гарантии доставки. Подходит для некритичных событий.
- TCP/514** → Надёжная доставка, но без шифрования.
- TLS/6514** → Надёжная доставка и шифрование. Рекомендуется для продакшена.

ПРИМЕР ПУТИ СООБЩЕНИЯ



ЧТО ВАЖНО ПОМНИТЬ

- Синхронизируйте время на всех узлах (NTP)
- Защитите каналы передачи (TLS, VPN, фаервол)
- Настройте ротацию и хранение журналов
- Контролируйте доступ к журналам

Централизация логов повышает видимость и безопасность инфраструктуры

4. Архитектура и конфигурация rsyslog в Linux

rsyslog (Rocket-fast system for log processing) — быстрая и модульная служба обработки логов

В Debian, Ubuntu и RHEL (Red Hat Enterprise Linux)

rsyslog часто доступен из стандартных репозиториях

rsyslog принимает локальные сообщения, читает входы, применяет правила и записывает или пересылает результат

По сравнению со старым syslogd rsyslog поддерживает модули, фильтры, TCP, TLS (Transport Layer Security) и очереди

Конфигурационные файлы rsyslog

`/etc/rsyslog.conf` — главный файл конфигурации службы

`/etc/rsyslog.d/` — каталог для отдельных пользовательских правил

Отдельный файл в `rsyslog.d` удобнее сопровождать, чем править весь главный конфиг

После изменения конфигурации нужно проверить синтаксис и перезапустить или перечитать службу

Inputs, rules и outputs

Input, вход, получает сообщение из файла, сокета, сети или модуля

Rule, правило, фильтрует сообщение по facility, severity, хосту, программе или тексту

Output, выход, записывает сообщение в файл, отправляет по сети или передает в другое хранилище

Такая цепочка позволяет одинаково обрабатывать локальные и удаленные события

Модули `imudp`, `imtcp` и `omfwd`

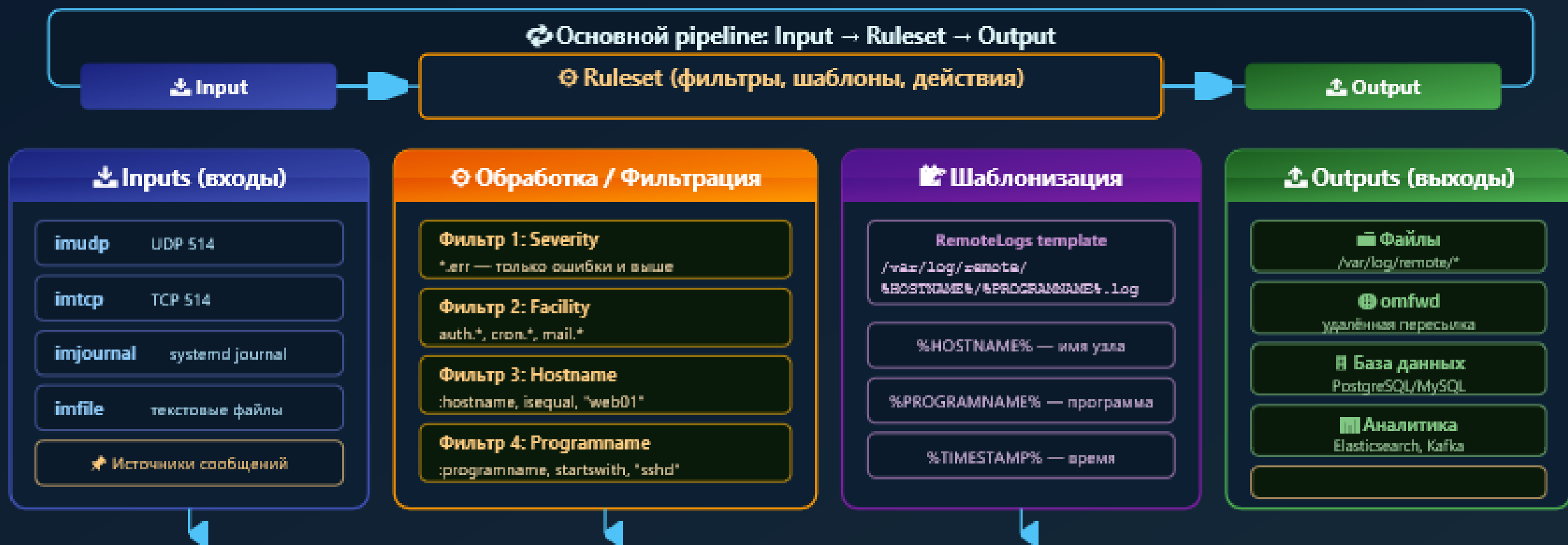
`imudp` — input module UDP, модуль приема Syslog-сообщений по UDP

`imtcp` — input module TCP, модуль приема Syslog-сообщений по TCP

`omfwd` — output module forward, модуль пересылки сообщений на удаленный сервер

Модули включаются явно, чтобы `rsyslog` понимал, какие входы и выходы использовать

Pipeline обработки сообщений rsyslog



PIPELINE ОБРАБОТКИ СООБЩЕНИЙ RSYSLOG

Как сообщение проходит путь от источника до файла или удалённого сервера



КАК ЭТО РАБОТАЕТ

- 1 rsyslog принимает сообщения из локальных и сетевых источников
- 2 складывает их во внутреннюю очередь
- 3 применяет правила и фильтры
- 4 выполняет нужное действие: записывает или пересылает
- 5 при необходимости использует буферизацию и повторную отправку

ЧТО ВАЖНО АДМИНИСТРАТОРУ

- ✓ Проверять inputs и открытые порты
- 📄 Тестировать конфигурацию: `rsyslogd -N1`
- 📁 Разделять локальные и удалённые логи
- </> Использовать templates и stop
- 🔒 Настраивать ротацию и контроль доступа



rsyslog = **входы** → **очередь** → **фильтрация** → **действие** → **хранение или пересылка**

5. Практика: настройка rsyslog-сервера-коллектора

Коллектор log01.company.test должен принимать сообщения по сети и сохранять их отдельно

В учебном сценарии log01 слушает UDP 514 и TCP 514

Удаленные сообщения сохраняются в /var/log/remote по имени хоста и программы

Перед включением приема нужно проверить firewall и права на каталог хранения

Подготовка каталога для удаленных логов

`sudo mkdir -p /var/log/remote` создает корневой каталог для удаленных журналов

`sudo chown syslog:adm /var/log/remote` назначает владельца, подходящего для записи rsyslog в Debian/Ubuntu

`sudo chmod 750 /var/log/remote` ограничивает доступ к журналам

Если rsyslog не может записать в каталог, сообщения будут теряться или попадать в локальные журналы ошибок

Прием логов по UDP и TCP

`module(load="imudp")` загружает модуль приема UDP-сообщений

`input(type="imudp" port="514")` включает прием Syslog по UDP 514

`module(load="imtcp")` загружает модуль приема TCP-сообщений

`input(type="imtcp" port="514")` включает прием Syslog по TCP 514

Эти строки обычно помещают в отдельный файл, например `/etc/rsyslog.d/10-remote-input.conf`

Динамический шаблон RemoteLogs

\$template

RemoteLogs, "/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log" задает путь хранения

%HOSTNAME% подставляет имя удаленного узла из сообщения

%PROGRAMNAME% подставляет имя программы, которая создала сообщение

. ?RemoteLogs записывает все facility и severity по этому шаблону

& stop прекращает дальнейшую обработку сообщения другими правилами

Зачем нужен & stop

& stop останавливает прохождение сообщения по следующим правилам rsyslog

Без stop удаленный лог может одновременно попасть в remote-каталог и локальные файлы log01

Это увеличивает объем журналов и усложняет анализ
stop особенно важен на коллекторах, где нужно четко отделять чужие события от собственных

Проверка конфигурации сервера

`sudo rsyslogd -N1` проверяет синтаксис конфигурации
`rsyslog` без запуска службы

`sudo systemctl restart rsyslog` применяет изменения
после успешной проверки

`sudo ss -tulnp | grep ':514'` показывает, слушает ли
сервер UDP или TCP порт 514

Если порт 514 не слушается, клиентские сообщения не
попадут на коллектор

НАСТРОЙКА log01 КАК RSYSLOG-КОЛЛЕКТОРА

Как сервер принимает удалённые Syslog-сообщения и сохраняет их по хостам и программам

1 РОЛЬ СЕРВЕРА log01



log01.company.test
192.168.10.100

- ✓ принимает журналы по сети
- ✓ слушает UDP 514 и TCP 514
- ✓ сохраняет удалённые логи отдельно
- ✓ использует rsyslog



2 1. ПОДГОТОВКА КАТАЛОГА

```
$ sudo mkdir -p /var/log/remote
```

→ создать каталог для удалённых журналов

```
$ sudo chown syslog:adm /var/log/remote
```

→ назначить владельца для записи rsyslog

```
$ sudo chmod 750 /var/log/remote
```

→ ограничить доступ к логам



/var/log/remote

3 2. ВКЛЮЧЕНИЕ ПРИЁМА ПО СЕТИ



/etc/rsyslog.d/10-remote-input.conf

```
module(load="imudp")
input(type="imudp" port="514")

module(load="imtcp")
input(type="imtcp" port="514")
```

- ✓ imudp — приём Syslog по UDP
- ✓ imtcp — приём Syslog по TCP
- ✓ порт 514 должен быть открыт в firewall

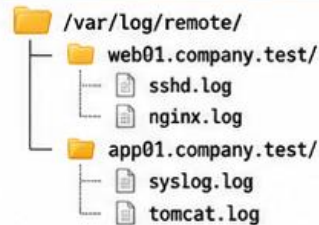
4 3. ШАБЛОН ХРАНЕНИЯ REMOTELOGS



/etc/rsyslog.d/20-remote-store.conf

```
$template RemoteLogs,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
*. * ?RemoteLogs
& stop
```

- %HOSTNAME%** → имя узла
- %PROGRAMNAME%** → имя программы
- & stop** → не даёт сообщению попасть в другие локальные правила



5 4. ПРИМЕНЕНИЕ И ПРОВЕРКА

```
$ sudo rsyslogd -N1
```



проверить синтаксис конфигурации

```
$ sudo systemctl restart rsyslog
```



перезапустить службу

```
$ sudo ss -tulnp | grep ':514'
```



убедиться, что сервер слушает порт 514

Netid	State	Recv-Q	Send-Q	Local Address:Port	Process
udp	UNCONN	0	0	0.0.0.0:514	users:("rsyslogd",pid=1234,fd=6)
tcp	LISTEN	0	128	0.0.0.0:514	users:("rsyslogd",pid=1234,fd=7)

6 ЧТО ВАЖНО НЕ ЗАБЫТЬ



открыть UDP/TCP 514 в firewall



хранить удалённые логи отдельно от локальных



использовать & stop, чтобы избежать дублирования



следить за местом на диске и logrotate



ограничить доступ к журналам



Syslog
UDP/TCP 514



log01
(rsyslog)



/var/log/remote/
%HOSTNAME%/
%PROGRAMNAME%.log

6. Практика: настройка rsyslog-клиента-отправителя

Клиент web01.company.test должен пересылать события на log01

Для простого стенда можно отправлять все события правилом *.*

В производственной сети часто фильтруют только нужные facility и severity

Для надежной передачи предпочтительно использовать TCP и очередь на клиенте

Традиционный синтаксис пересылки

. @192.168.10.100:514 отправляет все сообщения по UDP

. @@192.168.10.100:514 отправляет все сообщения по TCP

Один символ @ означает UDP, два символа @@ означают TCP

Такое правило можно поместить в /etc/rsyslog.d/60-forward.conf

После изменения файла требуется перезапуск rsyslog

Современный action omfwd

`action(type="omfwd" target="192.168.10.100"
port="514" protocol="tcp")` задает пересылку через
RainerScript

`target` указывает адрес коллектора log01

`protocol="tcp"` выбирает надежный транспорт

Такой синтаксис удобнее расширять очередями,
повторными попытками и TLS-параметрами

Буферизация на клиенте

`queue.type="LinkedList"` включает очередь сообщений в памяти

`queue.filename="forward_buffer"` задает имя файлового буфера на диске

`action.resumeRetryCount="-1"` заставляет rsyslog повторять отправку без ограничения числа попыток

`queue.saveonshutdown="on"` сохраняет очередь при остановке службы

Буферизация уменьшает потерю логов, если `log01` временно недоступен

Проверка клиента после настройки

`sudo rsyslogd -N1` проверяет синтаксис клиентской конфигурации

`sudo systemctl restart rsyslog` применяет правило пересылки

`logger -p local0.warn "test from web01"` создает тестовое Syslog-сообщение

На log01 команда `tail -f /var/log/remote/web01/*` показывает, пришло ли сообщение

ПЕРЕСЫЛКА ЛОГОВ WEB01 НА LOG01

Настраиваем rsyslog-клиент web01 для отправки логов на сервер-коллектор log01



1. УСЛОВИЯ



web01 — отправитель (клиент)
192.168.10.11



log01 — получатель (коллектор)
192.168.10.100



Протокол: UDP или TCP
Порт: 514



Синхронизированное время (NTP)
на обоих узлах

2. ПРОВЕРЯЕМ СЕТЕВОЕ СОЕДИНЕНИЕ

Проверяем, что web01 может достигать до log01.

Проверка UDP 514:

```
web01:~# nc -zuv 192.168.10.100 514
Connection to 192.168.10.100 514 port [udp/*] succeeded!
```

Проверка TCP 514:

```
web01:~# nc -zv 192.168.10.100 514
Connection to 192.168.10.100 514 port [tcp/*] succeeded!
```

3. РЕЗЕРВНАЯ КОПИЯ КОНФИГУРАЦИИ

Перед изменениями создаём резервную копию.

```
web01:~# sudo cp /etc/rsyslog.conf \
/etc/rsyslog.conf.bak.${date +%F_%H%M}
web01:~# sudo cp -r /etc/rsyslog.d \
/etc/rsyslog.d.bak.${date +%F_%H%M}
```

4. НАСТРАИВАЕМ ОТПРАВКУ ЛОГОВ НА LOG01

- 1 Открываем или создаём файл конфигурации отправки.

```
web01:~# sudo nano /etc/rsyslog.d/10-forward-to-log01.conf
```

- 2 Добавляем правила пересылки.

Отправим все сообщения (facility.*) на log01 по UDP.

```
# 10-forward-to-log01.conf
# Отправлять ВСЕ сообщения на log01 (UDP)
*,* @192.168.10.100:514

# (Альтернатива: по TCP - надёжнее, но медленнее)
# *,* @@192.168.10.100:514
```



Пояснения:

- *,* — все facility и все уровни severity
- @ — передача по UDP
- @@ — передача по TCP

- 3 (Опционально) Исключаем локальные записи, чтобы не дублировать их на log01.

Создаём файл исключений.

```
web01:~# sudo nano /etc/rsyslog.d/11-local-exclude.conf

# 11-local-exclude.conf
# Не отправлять локальные логи на сам localhost
if $hostname == 'web01.company.test' then stop
```

- 4 Проверяем синтаксис конфигурации.

```
web01:~# sudo rsyslogd -N1
rsyslogd: version ..., configuration OK
```

5. ПРИМЕНЯЕМ И ПРОВЕРЯЕМ

- 1 Перезапускаем rsyslog на web01.

```
web01:~# sudo systemctl restart rsyslog
```

- 2 Проверяем статус службы.

```
web01:~# systemctl status rsyslog
Active: active (running)
```

- 3 Генерируем тестовое сообщение.

```
web01:~# logger -p local0.info "Test from web01"
```



Проверяем на log01

На сервере-коллекторе сообщение должно появиться в файле удалённых логов.

```
log01:~# sudo tail -f /var/log/remote/web01.company.test/*.log
May 19 12:34:56 web01.company.test user: Test from web01
```

6. ПОЛЕЗНЫЕ КОМАНДЫ

- Посмотреть, куда будет отправлено сообщение:
web01:~# rsyslogd -N1
- Посмотреть очередь rsyslog:
web01:~# sudo rsyslogd -q
- Посмотреть активные правила:
web01:~# sudo rsyslogd -M
- Журнал службы rsyslog:
web01:~# journalctl -u rsyslog -f



7. СТРУКТУРА ФАЙЛОВ НА LOG01 (РЕЗУЛЬТАТ)

Логи web01 будут сохраняться в:

```
/var/log/remote/
├─ web01.company.test/
│  ├─ auth.log
│  ├─ syslog.log
│  ├─ cron.log
│  └─ local0.log # (например, сюда попадёт наше тестовое сообщение)
```



Преимущества:

- ✓ Централизация журналов
- ✓ Единое место хранения
- ✓ Удобный поиск и анализ
- ✓ Защита от потери логов при сбое узла

8. ВАЖНО ПОМНИТЬ

- ✓ Время на узлах должно быть синхронизировано (NTP)
- ✓ Для полной надёжности используйте TCP (@@)
- ✓ Не отправляйте логи на самовлано (stop), чтобы избежать циклов
- ✓ Следите за доступным местом на log01
- ✓ Регулярно проверяйте доставку (тест logger)



9. ВОЗМОЖНЫЕ ПРОБЛЕМЫ

- ✗ Нет соединения: проверьте сеть, порт 514 и фаервол
- ✗ Логи не приходят: проверьте конфиг, синтаксис (-N 1), статус rsyslog и тест logger
- ✗ Дублирование локальных логов: добавьте правило stop
- ℹ Потеря сообщений (UDP): используйте TCP для важных логов

7. logger и ротация логов logrotate

logger — консольная утилита для ручной отправки сообщения в систему логирования

logger полезен для тестов, Bash-скриптов и фиксации пользовательских событий

logrotate — утилита ротации журналов, которая переименовывает, сжимает и удаляет старые файлы

Без logrotate сервер-коллектор может потерять работоспособность из-за переполнения диска

Пример использования logger

`logger -p local0.warn "Тестовое предупреждение: сбой резервного копирования"` отправляет сообщение с facility `local0` и severity `warning`

Параметр `-p` задает пару `facility.severity`

Текст в кавычках становится телом Syslog-сообщения

На коллекторе это сообщение должно попасть в файл, соответствующий хосту и программе `logger`

Зачем нужна ротация логов

Журналы постоянно растут, особенно на центральном коллекторе

Ротация разделяет текущий файл и архивные файлы за прошлые периоды

Сжатие gzip уменьшает объем старых журналов на диске

Удаление старых архивов ограничивает срок хранения и защищает сервер от переполнения

Параметры logrotate

daily, weekly или monthly задают период ротации

rotate 14 означает хранить 14 архивных копий

compress включает сжатие старых журналов

missingok разрешает не считать ошибкой отсутствие файла

postrotate выполняет команды после ротации, например перезагрузку rsyslog

Проверка logrotate

`sudo logrotate -d /etc/logrotate.d/rsyslog` запускает dry-run без реального изменения файлов

Параметр `-d` показывает, что logrotate собирается сделать

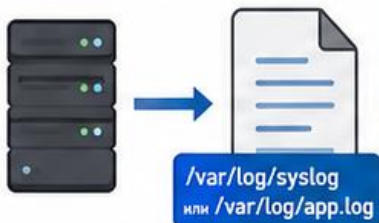
`sudo logrotate -f /etc/logrotate.d/rsyslog` принудительно выполняет ротацию для проверки

После теста нужно убедиться, что rsyslog продолжает писать в новый файл

ЖИЗНЕННЫЙ ЦИКЛ ФАЙЛА ЖУРНАЛА И LOGROTATE

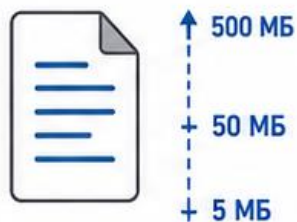
Как журналы растут, ротируются, архивируются и удаляются

1 Служба пишет в активный лог



- rsyslog, nginx, sshd и другие службы пишут в текущий файл
- Файл постоянно растёт по мере появления событий

2 Файл увеличивается



- Без контроля журналы занимают всё больше места
- На сервере-коллекторе рост особенно быстрый

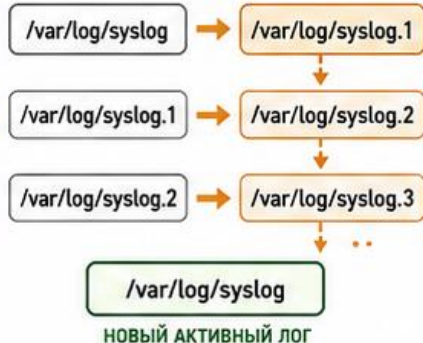
3 Срабатывает logrotate



- по расписанию: **daily / weekly / monthly**
- или по условию размера

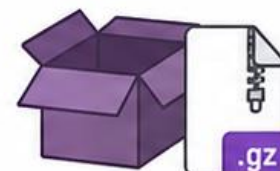
```
$ sudo logrotate /etc/logrotate.conf
```

4 Текущий файл ротируется



- текущий файл переименовывается
- создаётся новый активный лог
- служба продолжает писать уже в новый файл

5 Архивирование и сжатие



`/var/log/syslog.2.gz`
`/var/log/syslog.3.gz`
...

- старые журналы можно сжимать
- **compress** экономит место на диске

6 Удаление старых архивов



- хранятся только последние N копий
- старые архивы удаляются автоматически
- пример: **rotate 14**



активный лог



рост



ротация



новый файл



архив



удаление

ОСНОВНЫЕ ПАРАМЕТРЫ LOGROTATE



daily / weekly / monthly — период ротации



rotate 14 — число архивов



compress — сжатие



missingok — не считать ошибкой отсутствие файла



create — создать новый файл после ротации



postrotate — выполнить команду после ротации

ПОЧЕМУ ЭТО ВАЖНО



защита от переполнения диска



упорядоченное хранение журналов



удобство анализа и аудита



стабильная работа служб логирования

ПРОВЕРКА:



```
logrotate -d /etc/logrotate.d/rsyslog  
— тест без изменений
```



```
logrotate -f /etc/logrotate.d/rsyslog  
— принудительная ротация
```


8. Диагностика, тестирование и типичные ошибки

Диагностика начинается с разделения проблемы:
клиент не отправляет, сеть не пропускает или сервер не принимает

Затем проверяется, в какой файл `rsyslog` должен записать сообщение

После этого анализируют права на каталог, `stop`-правила и ротацию

Проверку лучше выполнять тестовым `logger`-сообщением с понятным текстом

Проверка синтаксиса и службы

`sudo rsyslogd -N1` проверяет конфигурацию до перезапуска службы

`systemctl status rsyslog` показывает, работает ли служба

`journalctl -u rsyslog -n 50 --no-pager` показывает последние ошибки rsyslog

Ошибка синтаксиса в одном файле может помешать применению всей конфигурации

Проверка сетевой доступности

`sudo ss -tulnp | grep ':514'` проверяет, слушает ли log01 порт 514

`nc -vz 192.168.10.100 514` проверяет TCP-доступность порта с клиента

Для UDP полезно проверять firewall и факт появления тестового сообщения на коллекторе

Если включен `ufw` или `nftables`, нужно разрешить выбранный протокол и порт

Проверка доставки на коллекторе

`tail -f /var/log/remote/web01/logger.log` показывает новые сообщения logger от web01

Если каталог с именем хоста не появился, нужно проверить HOSTNAME в сообщении и шаблон RemoteLogs

Если файл есть, но пустой, нужно проверить фильтр, stop-правило и права на запись

Если сообщение попало в локальный `/var/log/syslog`, значит правило удаленной раскладки не сработало как ожидалось

Типовые ошибки администратора

Забыли выполнить `systemctl restart rsyslog` после редактирования конфигурации

Firewall блокирует UDP или TCP 514 на сервере-коллекторе

Не настроен `& stop`, поэтому удаленные сообщения дублируются в локальные файлы

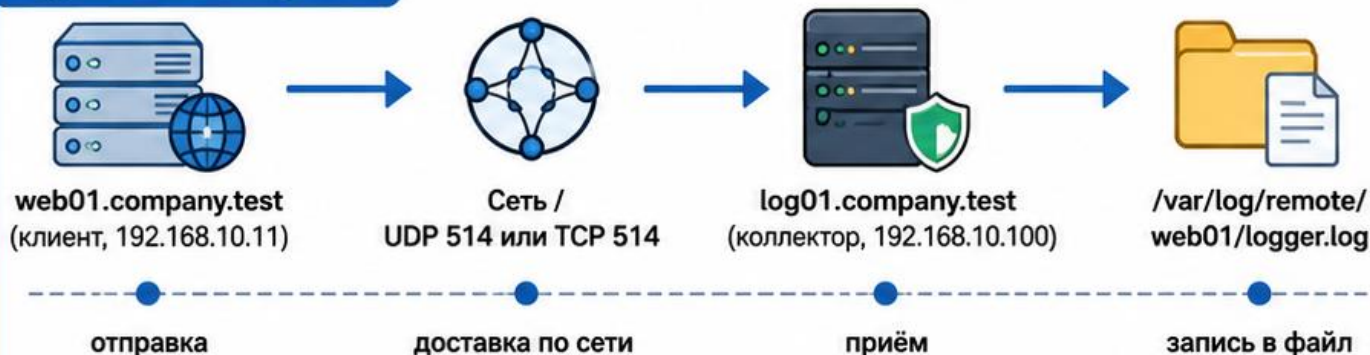
Нет `logrotate`, и диск коллектора постепенно заполняется журналами

Клиент отправляет UDP, а сервер слушает только TCP

ДИАГНОСТИКА ДОСТАВКИ SYSLOG-СООБЩЕНИЯ

Как проверить путь сообщения от клиента до сервера-коллектора.

1. ПУТЬ СООБЩЕНИЯ



2. ПРОВЕРКА НА КЛИЕНТЕ web01



- ✓ проверить синтаксис:
`sudo rsyslogd -N1`
- ✓ проверить службу:
`systemctl status rsyslog`
- ✓ проверить правило пересылки:
`*.* @@192.168.10.100:514`
- ✓ отправить тест:
`logger -p local0.warn "test from web01"`

если ошибка синтаксиса — сообщение не уйдёт.

3. ПРОВЕРКА СЕТИ



- на сервере проверить прослушивание:
`ss -tulnp | grep ':514'`
- с клиента проверить TCP:
`nc -vz 192.168.10.100 514`
- проверить firewall / ufw / nftables
- убедиться, что клиент и сервер используют один и тот же транспорт: UDP или TCP



порт закрыт



не тот протокол

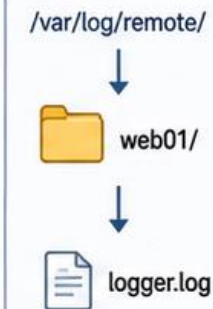


блокировка firewall

4. ПРОВЕРКА НА КОЛЛЕКТОРЕ log01



- проверить syntax и службу rsyslog
`sudo rsyslogd -N1, systemctl status rsyslog`
- проверить наличие каталога:
`/var/log/remete/web01/`
- смотреть файл в реальном времени:
`tail -f /var/log/remete/web01/logger.log`
- если сообщение попало в локальный `/var/log/syslog`, проверить шаблон RemoteLogs и правило «& stop»



5. ГДЕ ИСКАТЬ ПРИЧИНУ

Симптом	Что проверить
Сообщение не отправляется	конфиг клиента, <code>rsyslogd -N1, status rsyslog</code>
Порт недоступен	<code>ss, nc, firewall, маршрут</code>
Сообщение пришло, но не в тот файл	template RemoteLogs, <code>%HOSTNAME%, %PROGRAMNAME%</code>
Сообщение дублируется	правило & stop
Логи иногда теряются	UDP, перегрузка, лучше TCP и очередь

АЛГОРИТМ ДИАГНОСТИКИ



СОВЕТ: Лучше использовать понятное тестовое сообщение, например:

`logger -p local0.warn "test from web01"`

Итоги лекции

Централизованный сбор логов повышает наблюдаемость и сохраняет события при проблемах на отдельных серверах

Syslog классифицирует сообщения через facility и severity, а rsyslog реализует гибкую обработку в Linux

Коллектор должен принимать сообщения, раскладывать их по понятной структуре и защищать доступ к журналам

Клиентская пересылка может быть простой через @ или @@ либо надежной через omfwd и очередь

Стабильная работа логирования требует проверки

Контрольные вопросы

Почему централизованный сбор логов важен для информационной безопасности?

Что означают facility и severity в Syslog-сообщении?

Чем отличается отправка логов по UDP 514 и TCP 514?

Какие роли выполняют log-клиент, relay и сервер-коллектор?

Зачем в шаблоне RemoteLogs используются %HOSTNAME% и %PROGRAMNAME%?

Контрольные вопросы: настройка и диагностика

Чем отличаются правила `*.* @192.168.10.100:514` и `*.* @@192.168.10.100:514`?

Для чего используется символ `& stop` в конфигурации `rsyslog`?

Как с помощью `logger` отправить тестовое сообщение?

Почему `logrotate` критически важен для сервера-коллектора?

Что проверить, если сообщение не появляется в `/var/log/remote` на `log01`?